Adaptive Path-Tracking Controller Embedded With Reinforcement Learning and Preview Model for Autonomous Driving

Qi Xia, Peng Chen, *Member, IEEE*, Guoyan Xu, Haodong Sun, Liang Li, and Guizhen Yu

***Abstract*—Path tracking control is a crucial function in an autonomous vehicle. Previous studies have presented conventional model-based controllers, which lack generalization ability, especially in dealing with high-curvature cases. In contrast to those model-based controllers, data-driven controllers, represented by reinforcement learning (RL), are promising to improve tracking control accuracy and generalization by online learning. However, the inherent inexplicability of RL degrades the stability of an RL-based controller, further limiting its applications in real-world autonomous driving cases. To resolve this issue, this study proposes an adaptive tracking controller based on RL, allowing to enhance network stability through a preview model and adaptive correction. It is composed of three modules, namely a lateral controller, a lateral adaptive corrector, and a longitudinal speed planner. Specifically, the preview control theory is combined with the twin delayed deep deterministic policy gradient (TD3) method to improve the tracking convergence of the lateral controller; the adaptive corrector derives correction compensations based on predicted waypoints along a path, thus enhancing control accuracy; the longitudinal planner outputs a speed profile via a reward increment soft actor-critic algorithm, through which large lateral tracking errors are reduced. Using an intelligent vehicle with sufficient steering capability, simulations and real-world tests show that curvy paths with a curvature larger than 2.0 m^{-1} can be closely tracked within an error bound of 0.1 m.**

***Index Terms*—Tracking control, reinforcement learning, preview control, autonomous driving.**

I. Introduction

AUTONOMOUS driving technologies have made much progress in key domains including logistics, military, and rail transportation [1], [2]. Despite that autonomous vehicles have demonstrated reliable perception and decision-making capabilities, closed-loop tracking control remains essential for adhering to the demands of diverse operational strategies [3]. This necessitates a controller that can precisely and consistently execute smooth and safe maneuvers across a range of scenarios and various working conditions [4].

Numerous studies have focused on the tracking control issue of an autonomous vehicle, with the preview model serving as the foundation for traditional control algorithms. For instance, the Proportional Integral Derivative (PID) controller [5] minimizes the preview error, which is featured by being simple and stable in various applications. However, the optimal weighting parameters in a PID controller would vary under different working conditions, which reduces the tracking adaption if the weighting parameters are fixed. Simple geometric algorithms, such as the Pure-pursuit controller [6] and the Stanley controller [7], determine the tracking error using a look-ahead waypoint, offering flexibility to adjust the tracking performance to paths/trajectories of various curvatures. However, these methods can only address ideal vehicle models. Optimal-control-based controllers such as the Linear Quadratic Regulator (LQR) [8] and Model Predictive Control (MPC) [9] are based on the vehicle kinematic/dynamic model and optimal control theories. Crucially, the main component of their cost functions is derived from the error computed by the preview model, integrating a predictive element that enhances their effectiveness in dynamic environments. However, the control accuracy might be impacted for LQR and MPC when solving nonlinear problems, and time consumption in complex scenarios is also an issue [10].

In recent years, neural network-based control has emerged as a prominent research area [11]. Unlike conventional controllers, multi-layer network structures enable more nonlinear control characteristics, thereby enhancing the tracking adaptability across diverse scenarios while reducing the reliance on parametric fine-tuning. Among the existing neural network-based controllers, reinforcement learning (RL) distinguishes itself by not depending on pre-existing data for training [12]. End-to-end control is one of the mainstream research areas in this field, where the data collected from the perception module is fed into the neural networks to directly generate control signals. This method can directly handle the multi-dimensional input from perception sensors, reducing intermediate links. However, the training of neural networks is often marked by inexplicability

Received 24 July 2024; revised 11 October 2024 and 8 November 2024; accepted 17 November 2024. Date of publication 9 December 2024; date of current version 5 March 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 52272327 and in part by the Key Project of Beijing Municipal Institute of City Planning and Design under Grant 2024020266-13. The review of this article was coordinated by Prof. Zhaolong Ning. *(Corresponding author: Peng Chen.)*

Qi Xia, Peng Chen, Guoyan Xu, and Guizhen Yu are with the Key Laboratory of Autonomous Transportation Technology for Special Vehicles, Ministry of Industry and Information Technology, School of Transportation Science and Engineering, Beihang University, Beijing 100191, China (e-mail: xqi@buaa.edu.cn; cpeng@buaa.edu.cn; xuguoyan@buaa.edu.cn; yugz@buaa.edu.cn).

Haodong Sun is with the Beijing Municipal Institute of City Planning & Design, Beijing 100045, China (e-mail: haodongzs@163.com).

Liang Li is with the Beijing Municipal Commission of Planning and Natural Resources, Beijing 100045, China (e-mail: 18609132@qq.com).

Digital Object Identifier 10.1109/TVT.2024.3502640

0018-9545 © 2024 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on October 28,2026 at 12:11:49 UTC from IEEE Xplore. Restrictions apply.

and randomness, posing potential risks to the robustness of the control system.

Given the limitation of the learning-based controllers, combining network-based and model-based control has become a new research direction. For example, RL was used to adjust the parameters of the PID controller, and the neural network utilized as a fuzzy control parameter table [13]. In [14], RL was used to optimize the weight value of the QR matrix in MPC. However, the networks have been only used to adjust the parameters in the conventional model-based controllers so far. It indicates that the potential of the networks has not been fully leveraged yet.

The longitudinal speed will affect the lateral control of autonomous vehicles during cornering. Adapting the desired speed to road conditions can enhance control flexibility and accuracy. One way involves loading the desired path information for global speed planning [15]. Another way relies on the dynamic information at look-ahead waypoint as real-time speed constraints [16]. These methods effectively improve the traversing capabilities of autonomous vehicles in complex scenarios. However, integrating these with lateral control remains challenging, especially in dealing with efficiency (real-time performance) and longitudinal comfort.

To address these challenges, this study introduces an adaptive path-tracking controller that combines conventional control theories with RL, thus exploiting the strengths of both. The proposed controller is structured into three modules: a lateral controller, a lateral adaptive corrector, and a longitudinal speed planner. The lateral controller calculates the RL state and designs the corresponding reward function using a preview model. Subsequently, the twin delayed deep deterministic policy gradient (TD3) network generates the required yaw rate. The lateral adaptive corrector, employing a multi-point preview strategy, calculates and predicts the lateral deviation to refine the yaw rate. Meanwhile, the longitudinal speed planner adjusts speeds in real time based on tracking errors, using the reward increment soft actor-critic (RI-SAC) algorithm for adaptive integration with lateral control. The integration of RL with classical control theories not only mitigates the impact of longitudinal speed on lateral control but also achieves precise and adaptive path tracking for an autonomous vehicle.

The contributions of this study are outlined as follows:

1) A lateral control algorithm is proposed by integrating the TD3 network with a preview model. Compared to conventional controllers, this approach minimizes the need for frequent parameter adjustments while enhancing the portability of the RL training network.
2) A novel safe area exploration theory adapted to trajectory tracking is introduced, aimed at boosting the security and efficiency of the exploration process in RL. By optimizing the RL training process, and considering the rate of change in the reward reshaping function, the convergence speed of the RL network is enhanced.
3) An adaptive lateral deviation correction scheme is proposed. It calculates the lateral deviation according to the predicted trajectory, ensuring the timeliness of calculation and improving the control accuracy in application scenarios.
4) RI-SAC is deployed to output the speed adjustment factor, realize the adaptive matching of the lateral and longitudinal control volume, and improve the lateral control accuracy of curvy roads by taking the tracking control error as the input.

The remainder of this paper is structured as follows. Section II reviews related work on path tracking methods. Section III outlines the basic RL methodology employed in this study. Section IV details the principle of the proposed path tracking controller. Experimental results are reported and discussed in Section V, followed by conclusions drawn in Section VI.

II. Related Work

Path tracking controllers suitable for autonomous vehicles can be broadly categorized into two groups, namely the conventional control-theory-based methods and learning-based ones. Conventional controllers embedded with control theories typically include geometrics-based controllers, kinematics-based controllers, and dynamics-based controllers. Learning-based controllers utilize the robust data processing capabilities of neural networks to enhance the path tracking performance. The rest of this section reviews these different types of controllers.

A. Geometrics- and Kinematics-Based Controllers

For the tracking control of a reference path or trajectory, Ackermann vehicles typically use the Stanley controller or pure pursuit controller while engineering applications prefer to use the PID controller [5]. To enhance the performance of the classic control algorithms, researchers have incorporated curvature feedforward techniques [17]. Additionally, the adaptability of these algorithms has been improved through the usage of fuzzy PID controllers [18]. However, these controllers are overly sensitive to parametric settings and often perform far from being optimal [19].

In the realm of kinematics-based controllers, some researchers have derived smooth, time-varying commands based on motion prediction to facilitate tracking control in autonomous vehicles [20]. Others described the concerned tracking control scheme as an optimal control problem and solved it numerically via solving a quadratic programming problem [21]. Nevertheless, when external disturbances alter the motion characteristics of this scheme, its control effectiveness substantially deteriorates [22].

B. Dynamics-Based Controllers

In contrast to kinematic models, dynamic models describe the relationship between the vehicle motion states and control inputs more accurately. Combining LQR based on the dynamic model and the preview model to achieve optimal preview control is a classic method [23]. In [24], feedforward and LQR feedback control were combined to improve the tracking accuracy under extreme conditions, such as high-curvature and low adhesion. However, these methods are typically appropriate for stable working conditions with minor disturbances. Some researchers proposed updates to model parameters through error feedback by

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on October 28,2026 at 12:11:49 UTC from IEEE Xplore. Restrictions apply.

recognizing the time-varying characteristics of dynamic models, thereby enabling adaptive corrections in MPC [25] and leading to accurate steering system modeling [26]. Some researchers investigated robust MPC to tackle control challenges arising from uncertain systems [27]. Nevertheless, these solutions rely on precise dynamic models and demand substantial computational effort from the onboard computing unit.

In addition, sliding mode control and active disturbance rejection control were also used for path tracking control problems [28], [29]. However, most are still in the stage of theoretical research and rely on high-precision models.

C. Learning-Based Controllers

Learning-based control utilizes a nonlinear neural network to map the relationship between the state and control spaces. Unlike conventional controllers, a learning-based controller needs not to identify a precise and explicit model, thus is promising to handle complex and time-varying vehicle dynamics. Besides, learning-based controllers run fast when used online, thus allowing to react quickly to disturbances. Deep learning and deep reinforcement learning (DRL) are typical examples in this research field.

Deep learning strategies, such as imitation learning and inverse RL, rely on labeled data for training. A typical method of imitation learning uses expert experiences to train networks and achieve effective state-action mapping, as detailed in [30]. The inverse RL approach is discussed in [31]. Some studies such as [32] have integrated imitation learning with generative adversarial networks to reduce reliance on excessive expert data by updating training data online. Furthermore, guided cost learning [33] has been shown to enhance training efficiency significantly. However, the quality of training samples limits the potential of deep learning-based controllers, as not all the real data are good samples.

For DRL, end-to-end and hierarchical controls are the prevalent research directions. End-to-end control processes information from perception to execution seamlessly, thus integrating decision-making, open-loop planning, and closed-loop control. This end-to-end solution has been explored with visual sensor inputs [34] and multi-sensor fusion for obstacle avoidance [35]. However, multiple inputs would invoke uncertainties and disturbances. Hierarchical RL, in contrast, focuses on specific control challenges, such as tuning PID parameters via RL networks [36], showing advantages over the classical fuzzy PID methods. RL is also employed as a tool for predictive, time-domain adaptive optimization to enhance the robustness of MPC [37]. Nevertheless, it is essentially a conventional controller with inherent limitations in improving control accuracy. Several researchers directly applied RL to the control of differential vehicle [38]. The tracking control of Ackermann-steered autonomous vehicle has also been investigated [30]. The hierarchical control is effective in reducing input dimensions. However, the existing networks generally require more than 1000 rounds to converge. In response to this challenge, some researchers integrated imitation learning with RL [39], [40]. This approach enhances the convergence of the RL network. However, they rely on expert experience data and introduce certain challenges associated with network pretraining.

According to the previous studies, despite that an RL network can fit nonlinear error correspondences and enhance control precision, issues with convergence efficiency and algorithm adaptability still exist. These challenges make RL-based controllers ineffective for harsh cases in autonomous driving [41]. Thus, it is necessary to combine the advantages of traditional control theories and RL to improve control accuracy and network convergence efficiency, which will be the focus of this paper.

III. Background Knowledge of RL

This section presents the basic principle of RL. RL is guided by the reward function, and the strategy is optimized by gaining experience through the interactions between an agent and the environment. The exploration process is described by state s_t, action a_t, reward r_t, and the state at the next time step s_{t+1}. The resultant sequence is described in the form of a Markov chain $\{s_t, a_t, r_t, s_{t+1}, a_{t+1}, r_{t+1}, \cdots\}$. The agent needs to find the optimal strategy $\pi^*(a_t|s_t)$ in the environment that maximizes the overall expected return $R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$. The action value function estimates the value of expected return obtained by choosing a certain action for the current state, and it is updated by the Bellman's equations as:

$$Q_\pi(s_t, a_t) = \mathbb{E}_\pi \left[r_t + \gamma Q_\pi \left(s_{t+1}, a_{t+1} \right) | (s_t, a_t) \right] \quad (1)$$

where γ is a discount coefficient reflecting the importance of future rewards, $Q_\pi(s_{t+1}, a_{t+1})$ is the action value at the next moment, which is estimated by the network.

When the RL explores an optimal strategy, the action value function is maximized via:

$$Q^*(s_t, a_t) = \max_{\pi^*} Q_\pi(s_t, a_t) \quad (2)$$

Based on the above process, RL employs various methods to train optimal strategies, such as deep deterministic policy gradient (DDPG) [42], [43]. The TD3 and SAC [44], [45] algorithms used in this study are improvements upon DDPG, offering enhanced learning performance.

A. TD3

The TD3 algorithm is based on the actor-critic (AC) network framework [46]. Compared with the DDPG algorithm, TD3 is partially optimized to improve the training effect.

First, noise ϵ is added to the output of the target actor network $\mu(s_{t+1}; \phi^-)$ during training to make the predicted output Q value of the target critic network accurate.

$$\tilde{a}_{t+1} = \mu \left(s_{t+1}; \phi^- \right) + \epsilon, \epsilon \sim clip \left(\mathcal{N}(0, \sigma), -c, c \right) \quad (3)$$

where $\tilde{a}_t$ is the action after adding random noise, and σ, c are the constraint values for the random noise.

Second, two critic networks parameterized by θ, $Q_i(s_t, a_t; \theta_i)$ and $Q_i(s_t, a_t; \theta_i^-)$, are set up. The minimum Q value is selected as the result to prevent Q from being overestimated. The loss

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on October 28,2026 at 12:11:49 UTC from IEEE Xplore. Restrictions apply.

function is defined as:

$$L_i = \frac{1}{N}\sum\left(r_t + \gamma(1-d)\min_{i=1,2} Q_i\left(s_{t+1}, \tilde{a}_{t+1}; \theta_i^-\right) - Q_i\left(s_t, a_t; \theta_i\right)\right)^2 \quad (4)$$

where d is episode-ending indicator, N is the total number of data inputs to the network.

Finally, the target network is updated to accelerate the convergence by delaying the soft update.

$$\begin{aligned}\theta_i^- &= (1-\tau)\theta_i^- + \tau\theta_i \\ \phi^- &= (1-\tau)\phi^- + \tau\phi\end{aligned} \quad (5)$$

where τ is a hyperparameter that determines the update rate. The rest of TD3 is consistent with DDPG.

B. SAC

Like TD3, the SAC algorithm is based on the AC framework. Two schemes currently exist. In this study, the double target Q network, which is basically the same as the TD3 network structure, is selected. However, the SAC network considers the soft update of the action entropy $\mathcal{H}(\pi(.|s_t))$, which can make the final control effect robust. Its optimization strategy is

$$\pi^* = \arg\max_{\pi}\sum \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}\left[r(s_t,a_t) + \alpha\mathcal{H}(\pi(.|s_t))\right] \quad (6)$$

where α is the weight. The value network update formula structure is the same as the TD3 network structure. However, its network parameters are updated using the soft strategy:

$$J_Q(\theta) = \mathbb{E}_{(s_t,a_t)\sim D}\left[\frac{1}{2}\left(Q_{\theta soft}(s_t,a_t) - \left(r(s_t,a_t) + \gamma\mathbb{E}_{s_{t+1}\sim P}\left[V_{\bar{\theta} soft}(s_{t+1})\right]\right)\right)^2\right] \quad (7)$$

where D is the replay buffer, $Q_{\theta soft}(s_t,a_t)$ is the current action value, and $V_{\bar{\theta} soft}(s_{t+1})$ is the state value. Both values must be calculated by the maximum entropy.

The policy network update considers the action entropy, and the update basis is to minimize the KL divergence:

$$J_\pi(\phi) = \mathbb{E}_{s_t\sim D, a_t\sim\pi_\phi}\left[\log\pi_\phi(a_t|s_t) - \frac{1}{\alpha}Q_\theta(s_t,a_t) + \log Z(s_t)\right] \quad (8)$$

where $a_t = f_\phi(\varepsilon; s_t)$, ε is the random noise term. It determines the extent to which the output action adds random noise from a Gaussian distribution.

IV. Methodology

The overall research framework is illustrated in Fig. 1, consisting of three components: a lateral controller, a lateral corrector, and a longitudinal planner. The lateral controller utilizes a preview model to estimate tracking errors and generate lateral control commands. The lateral corrector adaptively adjusts the control output to diminish these tracking errors. The longitudinal planner processes both lateral and longitudinal errors as inputs to generate the desired speed, thereby achieving adaptive and stable tracking performances under various conditions. The rest of this section elaborates on the three modules: lateral control, adaptive lateral deviation correction, and adaptive speed planning.

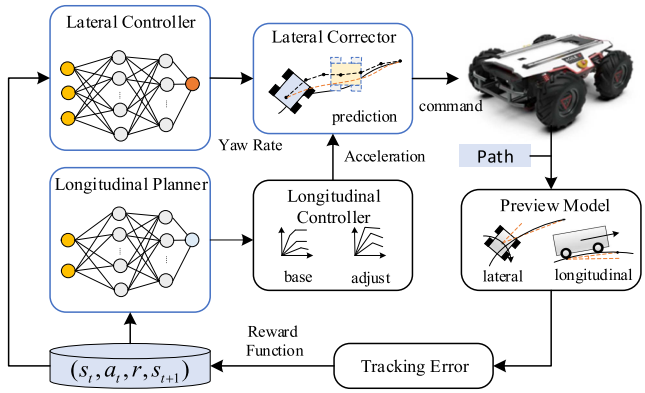


Fig. 1. Overall framework.

TABLE I
Description of Parameters

Category	Symbol	Description	Unit
Vehicle	l_f	Positioning center to front axle length	m
	b	Vehicle width	m
	l	Vehicle length	m
	m	Vehicle mass	kg
Motion	ω_r	Yaw rate	rad/s
	v_l	Longitudinal speed	m/s
	θ_n	Heading angle	rad
Reference path	c_m	Path key point curvature	m^{-1}
	θ_m	Path key point heading	rad
	c_p	Path look-ahead waypoint curvature	m^{-1}
Preview	e_d	Lateral deviation	rad
	e_α	Positioning angle deviation	m
	e_p	Predicted lateral deviation	m
	l_d	preview distance	m
RL	s_t^{lat}	Lateral controller status	-
	a_t^{lat}	Lateral controller action	rad/s
	r_t^{lat}	Lateral controller reward	-
	s_t^{lon}	Longitudinal planner status	-
	a_t^{lon}	Longitudinal planner action	-
	r_t^{lat}	Longitudinal planner reward	-
Controller	T	Control cycle	ms
	lat_t^{cmd}	Lateral command	rad/s
	lon_t^{cmd}	Longitudinal command	-

Table I introduces the key parameters in the methodology section, involving the categories related to the vehicle, motion, reference path, preview, RL and controller.

A. Preview-TD3 Lateral Controller

Firstly, we introduce the preview model utilized in this study. The tracking error calculated through this model serves as the

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on October 28,2026 at 12:11:49 UTC from IEEE Xplore. Restrictions apply.

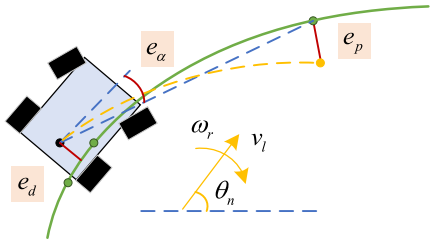


Fig. 2. Basic information of the preview model.

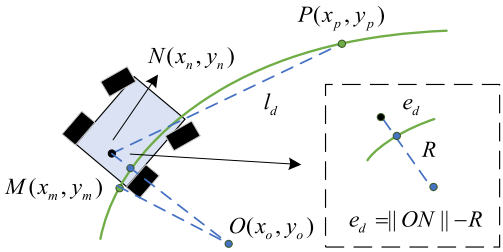


Fig. 3. Lateral deviation calculation.

input state for the RL network. Then, a safety exploration strategy is designed to mitigate the impact of network randomness and uncertainty on the convergence efficiency during the RL exploration process. Furthermore, the learning strategy is structured including the design of the state-action space and reward function.

1) Preview Model: The preview model, as one input and evaluation metric, directly influences the control outcome. Thus, the preview model needs to effectively reflect the tracking control performance of the autonomous vehicle. The relevant states used by the RL controller e_d, e_α, e_p and motion information ω_r, v_l, θ_n in this study are depicted in Fig. 2.

As shown in Fig. 3, the lateral deviation e_d is defined as the shortest distance from the current position point $N(x_n, y_n)$ to the reference path. As the reference trajectory is usually represented by discrete points, it requires to find the nearest path point $M(x_m, y_m)$ during the calculation. The coordinates of the center $O(x_o, y_o)$ obtained by solving the system of equations in (9) based on the geometric relationships.

$$\begin{cases} \tan\left(\frac{\pi}{2} + \theta_m\right) = \frac{y_0 - y_m}{x_0 - x_m} \\ \sqrt{(y_0 - y_m)^2 + (x_0 - x_m)^2} = \frac{1}{c_m} \end{cases} \tag{9}$$

where e_d is calculated as $||ON|| - 1/c_m$. If the autonomous vehicle is on the left side of the path (negative) or on the right side (positive), the formula for determining the sign of the lateral deviation is as:

$$e_d = \begin{cases} e_d \angle M'NM > 0 \\ -e_d \angle M'NM < 0 \end{cases} \tag{10}$$

where M' represents the next point on the reference path.

The angle deviation e_α includes both heading and lateral information and is represented by the angle between the line connecting the current point and the look-ahead waypoint$P(x_p, y_p)$:

$$e_\alpha = \arctan\left(\frac{y_p - y_n}{x_p - x_n}\right) - \theta_n \tag{11}$$

The predicted lateral deviation e_p reflects the effectiveness of the controller within the prediction time. The formula for calculating the predicted lateral deviation is

$$e_p = \left[l_d \sin e_\alpha - \frac{1}{2}\frac{\omega_r}{v_l}\left(|\overset{\frown}{MP}|\right)^2 \right]\left(1 - \frac{1}{2}\left(|\overset{\frown}{MP}| \cdot c_p\right)^2\right) \tag{12}$$

where $|\overset{\frown}{MP}|$ represents the reference path length between the nearest path points and the look-ahead waypoint, $l_d = ||NP||$.

Additionally, the preview model includes parameters such as the heading difference between the current position point and the nearest path point. However, these parameters have a limited impact on the control effectiveness and are not within the scope of this study's consideration.

2) Safety Exploration Strategy: During the RL training process, the instability of the initial network parameters may lead to the RL agent engaging in risky exploration in the environment. For the path tracking control, if the autonomous vehicle deviates too far from the main path, it could result in serious consequences in both training and control scenarios. Thus, it is necessary to assess the risks during the training process and design corresponding control strategies to ensure that the vehicle drives within safe boundaries in all situations.

In general, the reference path is considered as the safest region. During the control process, the autonomous vehicle is treated as a rectangular box, and the area swept by the vehicle when the lateral error is zero can be calculated, i.e., the minimum safe region. The lateral distance to the Left-Hand Side (LHS) of this area l_{left} is:

$$l_{left} = \begin{cases} b/2, & c_m < 0.005 \\ \sqrt{\left(\frac{1}{|c_m|} + \frac{b}{2}\right)^2 + l_f^2} - \frac{1}{|c_m|}, & c_m > 0.005 \end{cases} \tag{13}$$

where the curvature $c_m < 0$ indicates a left turn, while $c_m > 0$ indicates a right turn. The lateral distance to the Right-Hand Side (RHS) of this area l_{right} is:

$$l_{right} = \begin{cases} \sqrt{\left(\frac{1}{|c_m|} + \frac{b}{2}\right)^2 + l_f^2} - \frac{1}{|c_m|}, & c_m < -0.005 \\ b/2, & c_m > -0.005 \end{cases} \tag{14}$$

By referring to the established LHS and RHS boundaries $[l_{left}, l_{right}]$, the exploration regions can be divided into the safe exploration $[l_{left} + 0.5b, l_{right} + 0.5b]$ and non-exploration $[l_{left} + 0.8b, l_{right} + 0.8b]$, with corresponding parameters s_d, s_n as shown in Fig. 4. The region in which the vehicle runs is determined based on maximum lateral deviation $e_{\max}$.

Based on the defined exploration regions, a safety control strategy is designed. In the safe exploration region, the RL agent can explore through neural network control. In the safety control region, the safety controller is introduced, and the control inputs from both the neural network and the safety controller are

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on October 28,2026 at 12:11:49 UTC from IEEE Xplore. Restrictions apply.

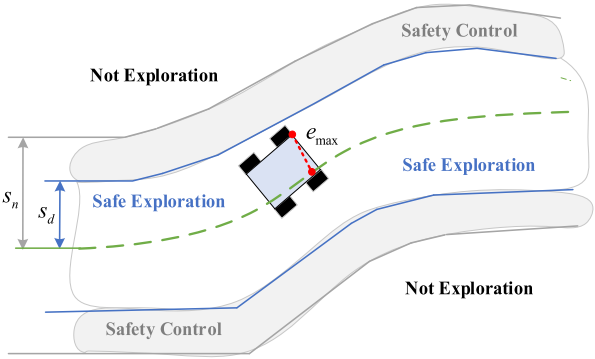


Fig. 4. Exploration area division.

saved in the memory. In the non-exploration region, the safety controller is fully engaged. These can be represented by the following expressions:

$$\omega_s = \begin{cases} 0, & |e_{\max}| \le s_d \\ \frac{1}{(s_n - e_{\max})^2 + 1} \cdot e_\alpha, & s_d < |e_{\max}| \le s_n \\ 2e_\alpha, & |e_{\max}| > s_n \end{cases} \tag{15}$$

where ω_s is the output of the safety control command.

When entering the non-exploration zone, a control strategy will also be adopted for the longitudinal speed. As shown in (16), the desired vehicle speed v_d will exponentially decay from$v_{d\max}$, gradually reducing to 0.

$$v_d = v_{d\max}\left(0.2 + \min\left(a_{d\max}, a_{d\max} e^{(s_n - e_{\max})}\right)\right) \tag{16}$$

where $a_{d\max} = 0.8$ represents the allowed variation range.

3) Learning Strategy for Lateral Controller: By integrating the preview model with RL methods, the exploration of the RL agent can be perceived as a self-optimizing process based on the preview model for path tracking control. The state space and action space correspond to the tracking state in the preview control theory, the reward function is also designed to minimize preview tracking error.

From the perspective of exploration, the vehicle trajectory in space is disordered. Theoretically, there exists an optimal quantity ω_r^* at each state that minimizes the error e_d at the next state $e'_{d\min}$, as shown in (17). However, since the network control input directly affects the position state of the agent, finding the optimal control input in a disordered space is challenging. Hence, the TD3 algorithm based on deterministic policy gradients is employed, to prevent random actions from contaminating the exploration process.

$$e'_{d\min} = e_d + \frac{1}{2} v_r \omega_r^* T^2 \tag{17}$$

Because the preview model can convert the complex positional relationship between the autonomous vehicle and the reference path into simple error values, the preview model is integrated to make the state inputs and reward function simpler and more explicit.

For the lateral controller training process, it is necessary to define the state s_t^{lat}, action a_t^{lat}, and reward r_t^{lat}.

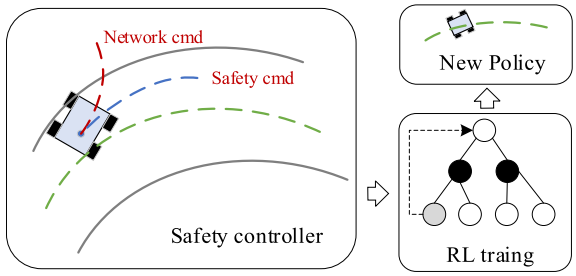


Fig. 5. The integration of RL and preview controller.

For the state s_{lat}, the angle deviation e_α in the preview model partially considers the road curvature and lateral deviation information, effectively reflecting the current tracking state. However, as pointed out in [47], during turns a single angle deviation control cannot effectively adjust the increasing lateral deviation, resulting in lateral deviation trend $\Delta\dot{y} > 0$. As shown in (18), the lateral deviation can be adjusted for trends.

$$\Delta\dot{y} = \frac{4\Delta s^2 - \alpha_0 e_d^2}{(L_d \cdot e_d)} \tag{18}$$

where Δs represents the path length from the key point to the look-ahead waypoint, and α_0 is an adjusting coefficient ensuring $\Delta\dot{y} < 0$.

Therefore, the lateral deviation e_d is introduced into the state. Simultaneously, the real-time yaw rate ω_r is input to guarantee the comfort requirement. Thus, the state input is determined as $s_t^{lat} = [e_\alpha, e_d, \omega_r]$, which effectively simplifies the RL control state inputs.

For the action a_t^{lat}, the output of the RL network is represented as ω_{dn}, which is constrained to the range of $[-1, 1]$. By adding random noise $\sigma_{lat}(t)$, the final RL network outputs $\omega_d \sim \mathcal{N}(\omega_{dn}, \sigma_{lat}(t))$. The size of $\sigma_{lat}(t)$ is adjusted to determine the magnitude of the output random noise, which will gradually become 0 after exploring for a certain period.

Furthermore, the proposed safety exploration strategy is integrated to constrain the action policies in the exploration space. As shown in Fig. 5, this approach restricts the agent to explore only within a bounded space. It helps simplify the training process and improve algorithm adaptability. Combining the lateral safety control value ω_s, action $a_t^{lat} = [\omega_d + \omega_s]$ will be stored in the memory buffer for network training.

For the reward function r_t^{lat}, three aspects are related including tracking evaluation, accuracy evaluation, and smoothness evaluation. The tracking evaluation resorts to the predicted lateral deviation e_p by the preview model, which characterizes the tracking performance. The accuracy evaluation is based on the lateral deviation e_d, whereas smoothness is assessed according to the angular velocity change rate $\triangle\ \omega_r$. The overall reward is expressed as

$$r_t^{lat} = e^{-5e_p^2} + \alpha_1 \times \left(\alpha_2 \times e^{-2(\Delta\omega_r)^2} + \alpha_3 \times e^{-5e_d^2}\right) + r_{end_lat} \tag{19}$$

where α_1, α_2, α_3 are the weighting coefficients, with values of 0.5, 1.0, 1.0, respectively. r_{lat_other} represents the state reward

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on October 28,2026 at 12:11:49 UTC from IEEE Xplore. Restrictions apply.

related to the episode, as shown in (20).

$$r_{lat_other} = \begin{cases} 1, & |e_{\max}| < s_d \text{ and } t_{end} = 1 \\ 0, & s_d < |e_{\max}| \le s_n \\ -e_d, & |e_{\max}| > s_n \end{cases} \tag{20}$$

where t_{end} represents the end of the path.

Since it is impossible to set a reference path with infinite length, there will exist a state transition when the episode is reset. To avoid its impact on network training, the state at the endpoint of the path is not stored in the RL replay buffer. The episode is reset when the vehicle reaches the end of the path or when it is outside the exploration area.

B. Self-Adaptive Lateral Corrector

The lateral controller trained allows to meet the task requirements under normal conditions. However, in scenarios with higher control accuracy requirements, e.g., running on the road with high-curvature, it poses a significant challenge for the controller. To address this issue, the network controller and kinematic model are used to predict the tracking state and design corresponding correction strategies. It will help further improve the lateral control accuracy.

To perform trajectory prediction, the motion model is built considering the motion characteristics of the autonomous vehicle, setting the initial predicted state $[x_k, y_k, \varphi_k]$:

$$\begin{cases} \dot{x}_k \cos\varphi + \dot{y}_r \sin\varphi = v_r \\ \dot{x}_k \sin\varphi - \dot{y}_k \cos\varphi = 0 \end{cases} \Rightarrow \begin{cases} \dot{x}_k = v_l \cos\varphi \\ \dot{y}_k = v_l \sin\varphi \\ \dot{\varphi}_k = \omega_r \end{cases} \tag{21}$$

By discretizing (21), the state transition matrix can be obtained as in (22). Then, the current RL state is input into the network controller to obtain the control quantity, which will be used in the state equation to estimate the position at the next moment $(x_{k+1}, y_{k+1}, \varphi_{k+1})$. Subsequently, based on the error tracking model, the RL state is updated, which facilitates the process of inputting it into the network to obtain the control quantity ω_k at the next moment. The number of iterations is determined by the set of prediction time.

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \\ \varphi_{k+1} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_k \\ y_k \\ \varphi_k \end{bmatrix} + \begin{bmatrix} \cos\varphi T & 0 \\ \sin\varphi T & 0 \\ 0 & T \end{bmatrix} \begin{bmatrix} v_l \\ \omega_k \end{bmatrix} \tag{22}$$

As shown in Fig. 6, a series of predicted discrete points are obtained based on the network controller and state equation. The calculation results of the lateral deviation $(e_1, e_2, \ldots, e_n)$ at each point reflect the tracking performance of the controller within the predicted time domain.

The lateral deviation of predicted waypoints cannot be directly used for control correction. The multi-point preview method from the preview theory is introduced and different weights are assigned to the deviations at predicted points according to the rules in [12]. Through multiplying the weights by their respective deviations and summing them up, the comprehensive lateral deviation of the predicted trajectory can be obtained for the controller, as shown in (23). It serves as the main control compensation quantity for adaptive correction.

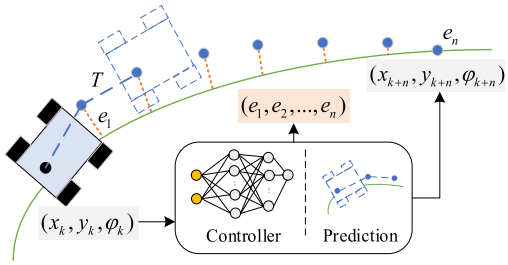


Fig. 6. Illustration of predictive state computation ($(x_{k+n}, y_{k+n}, \varphi_{k+n})$ represents the predicted position point after nT time).

TABLE II
WEIGHT SETTING RULES

Criteria	Coefficient determination
Consistent deviation signs	$\beta_1 = 1.5, \beta_2 = 0.0$
Same-side signs > 75%	$\beta_1 = 1.2, \beta_2 = 1.0$
Same-side signs > 50%	$\beta_1 = 1.0, \beta_2 = 0.5$
Both-side signs are equal	$\beta_1 = 1.0, \beta_2 = 0.0$

$$\omega_c = \beta_1 \omega_d + \sum_{i=1}^{n} \beta_2 e_i \tag{23}$$

where β_1 represents the proportional adjustment coefficient, and β_2 represents the integral adjustment coefficient. The magnitudes of these coefficients are shown in Table II.

Considering the external parameters such as the length l and the mass m of the vehicle, the action a_t^{lat} needs to be converted into real control commands denoted as $lat_t^{cmd} = F(\omega_c + a_t^{lat}, f(l, m \ldots))$.

C. Longitudinal Planner Based on RI- SAC

In complex environments with varying curved paths, the autonomous vehicle needs to perform tracking while ensuring operational efficiency by traveling at its maximum desired speed $v_{\max}$ (meeting the lateral physical limit conditions such as rollover limits). However, in scenarios with significant curvature, excessive speeds may result in increased lateral deviations or even make it impossible to perform tracking successfully. Thus, striking the right balance between planning speed and lateral control precision becomes crucial.

1) The Principle of RI-SAC: To enable the network to quickly learn longitudinal adjustment strategies, the Lyapunov stability theory is used to analyze the RL exploration process. In the context of speed planning, the quality of rewards corresponds to the goodness of actions. The training process can be taken as a procedure to maximize the rewards. Considering a Lyapunov scalar function $\mathcal{L}(s_t, a_t) \triangleq -r(s_t, a_t)$, during training the RL agent seeks better actions denoted as $\pi*$ such that the rewards gradually increase, i.e., $r(s_t, a_t) > r(s_{t-1}, a_{t-1})$. The following

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on October 28,2026 at 12:11:49 UTC from IEEE Xplore. Restrictions apply.

inequalities satisfies

$$\mathcal{L}(s_t, a_t) - \mathcal{L}(s_{t-1}, a_{t-1}) \leq 0 \tag{24}$$

In theory, as the agent explores its environment, there exists a maximum value $r*$ corresponding to the optimal strategy for the rewards. This means the scalar function $\mathcal{L}(s_t, a_t)$ should satisfy $\mathcal{L}(s_t, a_t) < -r*$ and $\mathcal{L}'(s_t, a_t) < 0$. According to the Lyapunov stability theory, the agent will converge to select the optimal strategy action $\pi*$ with $r*$. That is, the agent will eventually implement the optimal speed adjustment.

In previous study, the lateral controller utilizing the TD3 method achieved favorable results. However, speed planning needs to address the problem of lateral-longitudinal coupling. Deterministic policy gradients are subject to get stuck in local optima easily. Thus, this study adopts the SAC method which uses soft policy updates. By incorporating (6) into the maximum entropy calculation formula, the optimal policy is obtained as:

$$\pi^* = \arg\max_{\pi} \sum_t \mathbb{E}_{(s_t,a_t)\sim\rho_\pi} \left[r(s_t, a_t) - \alpha \log(\pi(.|s_t)) \right] \tag{25}$$

For the path tracking problem, relying on maximum entropy updates implies that there exist not a few random actions in the early stages of training. It cannot necessarily guarantee $\mathcal{L}(s_t, a_t) < \mathcal{L}(s_{t-1}, a_{t-1})$, which affects the convergence efficiency. To tackle it, we resort to stability analysis and redefine the Lyapunov scalar function as $\mathcal{L}_D(s_t, a_t) \triangleq -(r(s_t, a_t) + \Delta r_t)$. Near the maximum value $r*$, $\Delta r_t = r_t - \gamma^{-1} r_{t-1}$ is approximately equal to 0, meaning that the maximum reward remains mostly unchanged. As the reward increases, the agent will converge faster towards $r*$, which accelerates the convergence process. Thus, the reward increment function is redefined as:

$$r_s = r(s_t, a_t) + \eta \left(r(s_t, a_t) - \gamma^{-1} r(s_{t-1}, a_{t-1}) \right) / T \tag{26}$$

where η is the changing rate adjustment factor. By introducing the reward increment, the Q-value function required for updating the SAC action-value network is

$$\begin{aligned} Q^{\pi}_{soft}(s_t, a_t) = {} & r(s_t, a_t) \\ & + \eta[(r(s_t, a_t) - \gamma^{-1} r(s_{t-1}, a_{t-1})]/T \\ & + \lambda \mathbb{E}_{s_{t+1}, a_{t+1}} \\ & \left[Q^{\pi}_{soft}(s_{t+1}, a_{t+1}) - \alpha \log(\pi(a_{t+1}|s_{t+1})) \right] \end{aligned} \tag{27}$$

The action network update remains consistent with the original SAC approach, in which the new policy is updated proportional to the exponential function of Q by minimizing the KL divergence between the two distributions. Additionally, the policy is constrained to a Gaussian distribution within a bounded domain Π. The specific update policy is illustrated by (28), in which the Q-value calculation uses the reward increment formula.

$$\pi_{new} = \arg\min_{\pi' \in \prod} D_{KL} \left(\pi'(\cdot|s_t) \parallel \frac{\exp(Q^{\pi_{old}}(s_t, .))}{Z^{\pi_{old}}(s_t)} \right) \tag{28}$$

where $Z^{\pi_{old}}(s_t)$ is the partition function used to normalize the distribution.

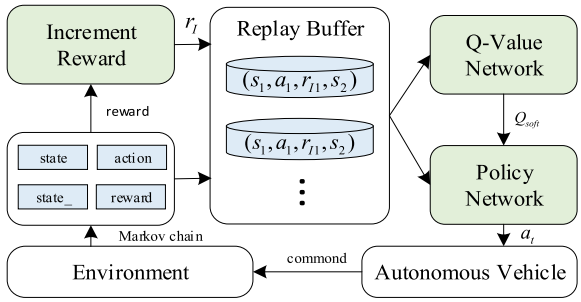


Fig. 7. The update process of RI- SAC network.

After the reward reshaping, the loss function for updating the state value network is calculated as:

$$\begin{aligned} & J_Q(\theta) \\ & = \mathbb{E}_{(s_t,a_t)\sim D} \left[\frac{1}{2} (Q_{\theta soft}(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1}\sim P}[V_{\bar{\theta} soft}(s_{t+1})] \right. \\ & \left. - \left(r(s_t, a_t) + \eta(r(s_t, a_t) - \gamma^{-1} r(s_{t-1}, a_{t-1}))/T \right)^2 \right] \end{aligned} \tag{29}$$

where $Q_{\theta soft}(s_t, a_t)$ and $V_{\bar{\theta} soft}(s_t, a_t)$ are updated based on the reshaped reward calculation.

Overall, the reward convergence process is accelerated towards the maximum value, and the early instability is reduced arising from the soft policy updates of the SAC algorithm. All these helps promote the rapid convergence of the agent to the optimal policy. The overall framework of this process is shown in Fig. 7.

2) Learning Strategy for Longitudinal Planner: When combining speed planning with RL, the network takes lateral and longitudinal tracking states as input and outputs a speed adjustment factor. If the lateral control algorithm computes a constant yaw rate, increasing the speed will decrease the equivalent front wheel steering angle δ^*, whereas decreasing the speed will have the opposite effect.

The equivalent angle is calculated using (30). Speed planning involves finding the optimal desired speed $v^*{}_{d\max}$ in the current state to achieve the optimal equivalent angle, thereby reducing lateral control errors or travel time.

$$\delta^* = \arctan \frac{l\omega_c}{v^*{}_{d\max}} \tag{30}$$

For the longitudinal planner training process, it is necessary to define the state s_t^{lon}, action a_t^{lon}, and reward r_t^{lon}.

For the state s_t^{lon}, given the lateral control state input $[e_\alpha, e_d, \omega_r]$, the goal of the longitudinal speed planner is to make the vehicle follow the maximum desired speed $v_{\max}$ as closely as possible. The longitudinal tracking error v_e can be represented as $(v_{d\max} - v_l)/v_{d\max}$, and the current speed v_n as $v_l/v_{d\max}$. Overall, the longitudinal network input can be represented as $s_t^{lon} = [e_\alpha, e_d, \omega_r, v_e, v_n]$.

For the action a_t^{lon}, the longitudinal network outputs a speed adjustment factor ξ, which is constrained to the range of $[-0.6, 0]$. To guarantee exploration randomness, a certain noise is introduced into the longitudinal output, as show in (31). Action $a_t^{lon} = [\xi_d]$. The speed adjustment factor adjusts

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on October 28,2026 at 12:11:49 UTC from IEEE Xplore. Restrictions apply.

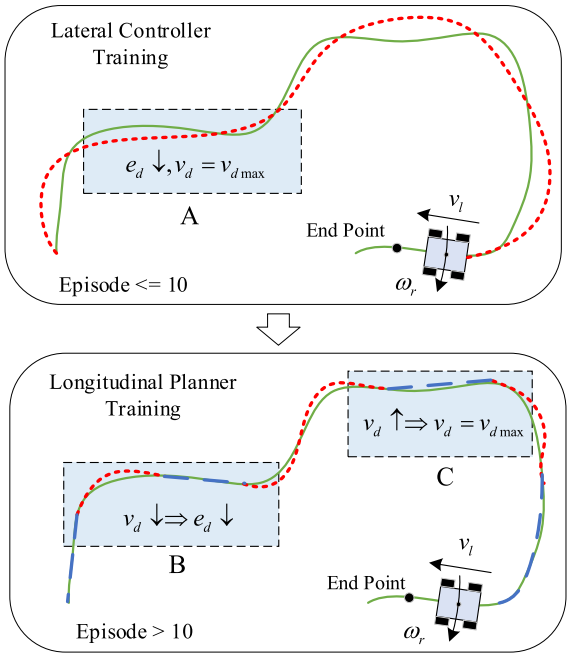


Fig. 8. The longitudinal training process.

the vehicle's speed by changing the desired vehicle speed $v_d = v_{d\max}(1+\xi_d)$.

$$\xi_d = \xi_{dn} + \kappa, \kappa \sim \mathcal{N}(0, \sigma_{lon}(t)) \tag{31}$$

where $\sigma_{lon}(t)$ represents the variance, and it decreases as the iterations increase. The initial noise is constrained within the range of $[-0.2, 0.2]$.

The desired speed can be represented by the function $v_d = F(s_t^{lat}, v_{d\max}, v_l)$. It is closely related to the lateral and longitudinal tracking performance.

For the reward r_t^{lon}, the objective of speed planning is to minimize the lateral control error and the longitudinal speed error. The tracking reward function can be represented as:

$$r_{track} = c_1 r_{lat} + (1-c_1)\, r_{lon} \tag{32}$$

where c_1 is used to adjust the weights of lateral and longitudinal components, and is set to 0.7 in this study, r_{lat} represents the lateral tracking reward $e^{-5\alpha^2}$, and r_{lon} represents the longitudinal tracking reward $e^{-5v_e^2}$.

Then, the final reward function can be obtained as:

$$r_t^{lon} = c_1 e^{-5\alpha^2} + (1-c_1)e^{-5v_e^2} - c_2|\Delta v_e| + r_{end_lon} \tag{33}$$

where c_2 is the smoothness coefficient, set to 0.5, $-|\Delta v_e|$ represents the control smoothness reward, and r_{end_lon} denotes the extra reward at the end of one episode. When the agent reaches the endpoint, the reward is set to 1.0.

The longitudinal training process is illustrated in Fig. 8. Process A is dedicated to lateral training, aiming to obtain a stable lateral controller. During the longitudinal training process, the lateral corrector is active. Processes B and C focus on longitudinal training by reducing lateral errors in curved sections and improving vehicle speed on straight sections.

TABLE III
THE PARAMETERS FOR LONGITUDINAL CONTROL

Parameter	Value	Parameter	Value
k_a	-0.000188	k_p	0.0565
k_b	0.020878	k_d	0.69
k_c	0.027858	k_s	0.2
k_s	1.1	k_f	0.95

When the agent reaches the end of the path, the environment is updated. The maximum travel time T_m at the lowest speed can be calculated based on the path length. To prevent training from getting stuck in a low-speed strategy, if the episode time exceeds the expected value $T_m + 10$, the episode will be reset. To avoid training from getting stuck in a high-speed strategy, if the lateral deviation exceeds 1.0 m, the episode is reset.

3) Longitudinal Controller: A feedforward-PID longitudinal controller is designed to track the desired speed $v_{d\max}$ of the vehicle. Three components are related, i.e., feedforward control, PD control, and integral control with a forgetting factor:

$$lon_{cmd} = u_v + u_s + u_{PD} + u_I \tag{34}$$

where $lon_{cmd} \in [-1, 1]$ denotes the output of the longitudinal controller, u_v estimated by a parametric quadratic function $u_v = k_a v_{d\max}^2 + k_b v_{d\max} + k_c$, represents the speed base term, u_s approximated by a parametric linear function $u_s = k_s i$, represents the slope i base term, u_{PD} and u_I are related to the speed error $e_v = v_{d\max} - v_l$, and respectively denote the adjustment terms for the PD and integral control according to (35). The related parameters for longitudinal control are determined through practical calibration using actual vehicle's dynamic characteristics, the determined parameters are shown in Table III.

$$\begin{cases} u_{PD} = k_p(e_v)_t + k_d(\Delta e_v)_t \\ u_I = k_i \times \left(k_f \int_0^{t-1} (e_v)_{t-1} dt + (e_v)_t\right) \end{cases} \tag{35}$$

D. Learning Process Description

The algorithm combines the lateral controller and longitudinal planner to facilitate the use of both algorithms. During training, the lateral TD3 and longitudinal SAC algorithms are trained simultaneously. When the agent has not yet reached the end of the path, actions a_t^{lat} and a_t^{lon} are updated based on lateral state input s_t^{lat} and longitudinal state input s_t^{lon}. The final control quantities lat_t^{cmd} and lon_t^{cmd} are calculated based on safety control strategies and predictive correction strategies, and rewards r_t^{lat} and r_t^{lon} are computed based on the environment feedback and new state. To update the memory buffer, it is necessary to check whether the state satisfies the update conditions. Subsequently, update the longitudinal SAC network every 10 episodes, while updating the lateral TD3 network before. The specific process is outlined in Algorithm 1.

When end-of-episode conditions are met, the vehicle state will be reset, and training restarts. The training frequency for both lateral and longitudinal control is 50 Hz.

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on October 28,2026 at 12:11:49 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Learning Process.

Initializing the TD3 and SAC network parameters.
Inputting parameters for PID and PP control algorithms.
for $i \in \{1, \ldots n\}$ **do**
  Initialize the state s_0^{lat}, s_0^{lon} and the replay buffer M_{lat}, M_{lon}.
  while (x_n, y_n) is not close to end point **do**
    $a_t^{lat}, a_t^{lon} \leftarrow$ select_action (s_t^{lat}, s_t^{lon})
    $lat_t^{cmd}, lon_t^{cmd} \leftarrow$ compute_action $(a_t^{lat}, a_t^{lon}, s_t^{lon}, (f, m \ldots))$
    $s_{t+1}^{lat}, s_{t+1}^{lon} \leftarrow$ compute_state (lat_{cmd}, lon_{cmd})
    $r_{lat_t}, r_{lon_t} \leftarrow$ compute_reward $(a_t^{lat}, a_t^{lon}, s_t^{lat}, s_{t+1}^{lat}, s_t^{lon}, s_{t+1}^{lon})$
    if $(s_t^{lon}[0] - s_t^{lon}[0] < 0.1)$ **and** $(s_t^{lon}[1] < 5.0$ **and** $s_t^{lon}[4] < 2.0)$ **then**
      push back M_{lat}, M_{lon}
    end
    update $s_{t+1}^{lat}, s_{t+1}^{lon}$
    if len $(M_{lat}) > 512$ **and** Episode $<= 10$ **then**
      update TD3 network
    end
    if len $(M_{lon}) > 512$ **and** Episode > 10 **then**
      update SAC network
    end
  end while
end for

V. Experimental Results and Discussions

This section describes the experimental validation of the proposed tracking controller. Considering safety and the feasibility of validation in large curvature scenarios, the RL training environment was established using ROS/Gazebo. To ensure the simulations can accurately reflected real-world conditions, the actual driving characteristics of the vehicle were integrated into the model. The training hardware included a CPU I9-12900KF and a GPU GeForce RTX 3080. After extensive training within this simulation environment, the setup was transitioned to real-world testing. The vehicle used was the line-controlled chassis, measuring 1080 mm × 726 mm × 336 mm, equipped with an Nvidia-Xavier onboard computing unit. In both simulation and real-world environments, the intelligent car can navigate curves with a curvature greater than 2m^{-1}.

A. Environment Setup

A path planner was deployed to generate reference paths for the controller to track. The reference path consists of sequenced waypoints in every 0.1m, adhering to a predefined maximum curvature constraint. The controlled object is a wheeled mobile robot in the Gazebo simulation environment, capable of receiving corresponding lateral and longitudinal commands. Details of the simulation environment are listed in Table IV.

At the early stages of lateral controller training, the use of a safety controller could make the maximum error within the range of 1m. When training the longitudinal controller initially, the desired speed v_d was randomized in $[0, v_{d\max}]$. If the lateral error of the autonomous vehicle exceeds 1.5m, or it maintains the minimum speed over 2s, the environment would be reset.

TABLE IV
Simulation Environment Configuration

Name	Illustration	Version
Linux	Operating System	Ubuntu 20.04
ROS/Gazebo	Runtime Environment	Noetic
Cuda	Computational Software	11.4
Python/C++	Programming Language	3.7.4/11
Pytorch	Network Framework	1.12.1

TABLE V
RL Training Parameter Settings

Network	Learning rate	Size of memory	Batch size	Soft-update rate	Soft-update weight
Pre-TD3	0.001	3500	512	0.01	/
RI-SAC	0.002	3500	512	0.01	0.05

B. Training Results

Based on the proposed approach, training was conducted separately for the controller and planner. Both networks share the same architecture, with differences only in inputs and outputs. The networks have a hidden layer with 512 neurons, using ReLU activation functions and a Tanh activation function in the output layer. Training parameters were adjusted based on convergence results. Table V provides the parameter configurations for lateral and longitudinal training.

Each training episode consists of a maximum of 1000 iterations, and the training ends when the reward changes within the range of ±0.05 over 10 episodes. The control cycle was set to 50 milliseconds. To validate the training results, two lateral control scenarios were selected for training: normal curvature (<0.5 m^{-1}) and high-curvature (<1.0 m^{-1}), as shown in Fig. 9(a) and (b). For longitudinal speed planner training, two scenarios were related: Single-turn steering and Continuous-turn steering, as illustrated in Fig. 9(c) and (d). After completing both lateral and longitudinal training, the average reward function at the end of each episode was recorded as the comprehensive reward for the episode.

For lateral control training, the training results of three approaches were compared, i.e., Pre-TD3, TD3, and SAC, to investigate the performance of the deterministic policy gradient in lateral control as well as the role of the preview model in training. For longitudinal speed planner training, the results of RI-SAC, SAC, and TD3 were compared in the specific curvature scenario. The autonomous vehicle cannot pass at maximum speed under this curvature. It aimed to verify the performance of the soft policy update in multi-objective optimization problems and examine the effect of introducing reward increment on training efficiency.

Fig. 10 shows the training results for the lateral controller. In the case of normal curvature scenarios, the SAC algorithm

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on October 28,2026 at 12:11:49 UTC from IEEE Xplore. Restrictions apply.

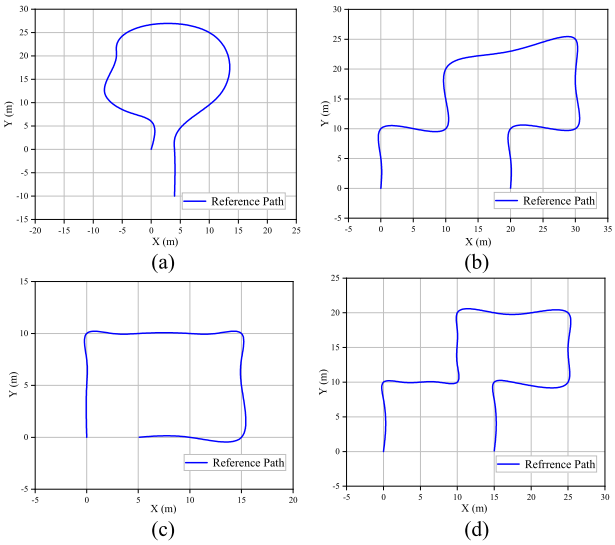


Fig. 9. RL training scenarios configuration. (a) Normal curvature. (b) High-curvature. (c) Single-turn steering. (d) Continuous-turn steering.

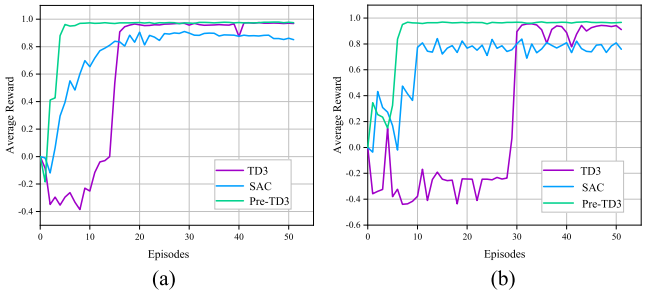


Fig. 10. Lateral control training results. (a) Normal curvature path. (b) High-curvature path.

converges in about 20 episodes but fails to reach the maximum reward. The TD3 algorithm converges in 15 episodes and achieves the maximum reward. By comparison, the Pre-TD3 algorithm converges in 6 episodes and achieves the maximum reward. In high-curvature scenarios, the SAC algorithm converges in approximately 30 episodes but fails to reach the maximum reward. The TD3 algorithm converges in 10 episodes, achieves the maximum reward but exhibits oscillations. By comparison, the Pre-TD3 algorithm converges in 8 episodes, and remains stable after attaining the maximum reward. Taken together, Pre-TD3 demonstrates superior convergence efficiency and stability compared to TD3 and SAC.

Fig. 11 illustrates the training results for longitudinal speed planner using various RL methods. It reveals that TD3 and RI-SAC achieves convergence more quickly than SAC in both scenarios. Furthermore, RI-SAC achieves a higher maximum reward than TD3 and SAC when achieving convergence, confirming the effectiveness of the reward increment design.

C. Simulation Testing Results

The control effectiveness of the vehicle path tracking was mainly measured by the lateral deviation from the reference path. In Simulation, a different reference path from the training

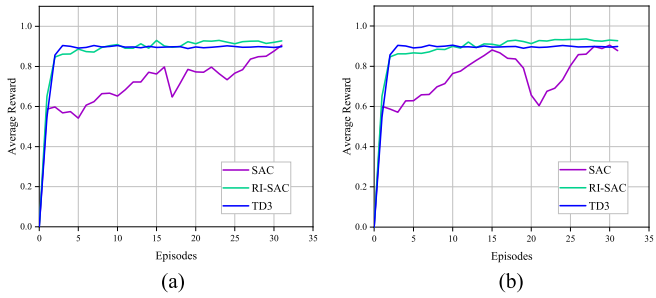


Fig. 11. Longitudinal speed planning training results. (a) Single-sided steering path. (b) Dual-sided steering.

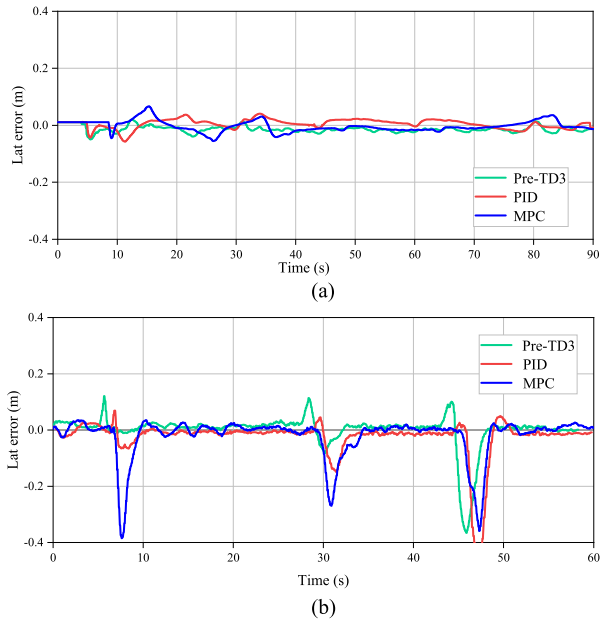


Fig. 12. Comparison of lateral control performance of MPC, PID and RL. (a) Normal curvature path. (b) High-curvature path.

scenario was set to validate the algorithm performance. After verifying the lateral controller with the network parameters saved during RL training, the speed planner effectiveness was validated in scenarios with high-curvature where the vehicle cannot track the path smoothly.

Lateral controllers were compared among PID, kinematic MPC, and Pre-TD3 algorithms. Parameter settings can affect the control performance. The parameters of MPC and PID are adjusted to achieve a lateral deviation of less than 0.1m in normal curvature scenarios and less than 0.5m in high-curvature scenarios. Then, the controllers using these parameters will be compared with the RL. Fig. 12(a) illustrates the control performance of these algorithms in the normal curvature scenario. The Pre-TD3 algorithm achieves a maximum deviation less than 0.05 m, which evidently outperforms PID and MPC algorithms. In the high-curvature scenario, the Pre-TD3 algorithm performs stably with a maximum deviation less than 0.35 m. By comparison, MPC and PID algorithms have deviations over 0.35 m, as shown in Fig. 12(b). These results verify that the RL algorithms with non-linear network fitting are superior to traditional algorithms.

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on October 28,2026 at 12:11:49 UTC from IEEE Xplore. Restrictions apply.

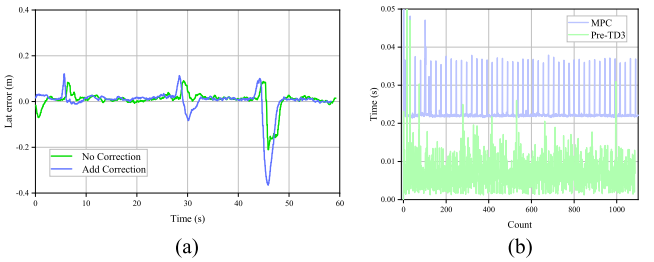


Fig. 13. The performance of adaptive correction control. (a) Lateral deviation after correction. (b) MPC computation time.

Furthermore, the lateral control accuracy could be further improved by incorporating adaptive correction under certain curvature conditions. For example, in the high-curvature scenario as shown in Fig. 12(b), after adding predictive correction, as shown in Fig. 13(a), the root-mean-square lateral deviation decreased from 0.08 m to 0.05 m, and the maximum lateral deviation decreased from 0.35 m to 0.2 m, resulting in approximately 40% improvement in control accuracy.

During each algorithm execution, 10 control quantities will be computed for adaptive correction. This increases the computation time of the algorithm. However, due to the efficiency of reading the pre-trained network parameters, the introduction of predictive correction still performs more efficiently than MPC quadratic programming solver. As shown in Fig. 13(b), with the introduction of predictive correction, the computation time increases from 5ms to around 10ms, but it remains significantly lower than 20ms by MPC.

Based on the pre-trained lateral control network, the longitudinal speed planner was validated in scenarios with curvature larger than 1.8m^{-1}. In such case, it is impossible for the vehicle to navigate at the maximum speed. The RL network output a speed adjustment factor to adjust the desired speed, and longitudinal speed tracking was achieved through the PID control. Fig. 14 shows the speed planning and corresponding lateral control effects. In scenarios with normal curvature, the desired speed was generally consistent with the maximum speed. In scenarios with high-curvature, an appropriate deceleration was applied to improve lateral control accuracy. Fig. 14(a) compares the desired speeds implemented by SAC and RI-SAC. It can be found that SAC leads to decelerating more frequently, thus affecting efficiency, while RI-SAC corresponds to more reasonable decelerations. Fig. 14(b) shows the corresponding lateral deviation under speed planning. It reveals that using RI-SAC for speed planning helps reduce lateral deviation within 0.1 m.

The simulation experiment results are summarized in Table VI. For the lateral controller, the maximum lateral deviation of the RL is less than 0.05m, and the average lateral deviation is less than 0.016 m, which is significantly better than MPC and PID. For the longitudinal speed planner, the method proposed in this study resulted in a maximum lateral deviation of less than 0.1 m. Furthermore, the passing speed obtained by the proposed method was higher than that by the method combining PID and the longitudinal speed planner in the same scenarios.

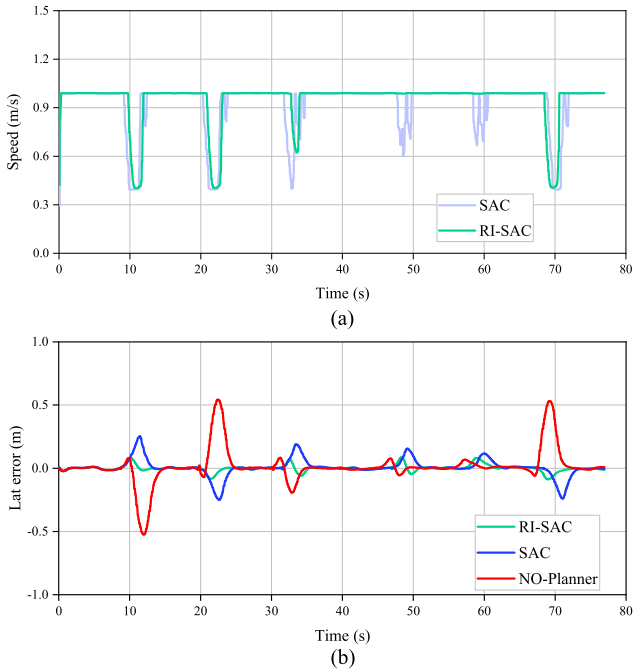


Fig. 14. The impact of speed planning on lateral deviation. (a) Effect of longitudinal speed planning. (b) Lateral control deviation.

TABLE VI
SIMULATION EXPERIMENT RESULTS

	Algorithm	Max c_m (m^{-1})	Average e_d (m)	Max e_d (m)	Average speed (v_{max})
LAT	PID	0.5	0.014	0.058	$0.988v_{max}$
	MPC	0.5	0.016	0.066	$0.986v_{max}$
	RL	0.5	0.016	0.050	$0.989v_{max}$
LON	NO+ LAT-RL	1.0	0.043	0.366	$0.974v_{max}$
	LON-RL+ PID	1.0	0.039	0.105	$0.947v_{max}$
	LON-RL+ LAT-RL	1.0	0.026	0.094	$0.923v_{max}$
	LON-RL+ LAT-RL	1.8	0.025	0.072	$0.878v_{max}$



Fig. 15. The testing platform and real-world testing scenario.

D. Real Test Results

We selected the network model exhibiting the most desirable tracking performance in the simulation environment and deployed it on a mobile robot for validation. The experimental platform for the mobile robot was built on the Nvidia Xavier onboard computing unit and integrated modules such as localization and wire, as shown in Fig. 15. Based on the experimental

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on October 28,2026 at 12:11:49 UTC from IEEE Xplore. Restrictions apply.

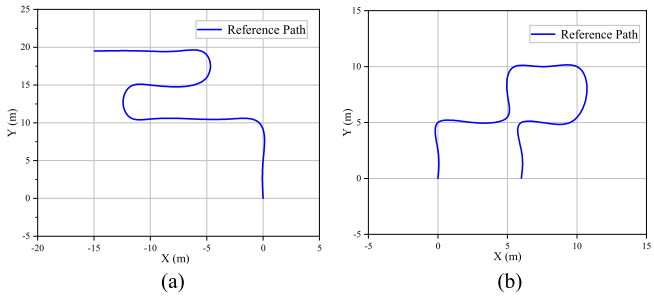


Fig. 16. Planned paths in real testing environment. (a) Normal curvature. (b) High curvature.

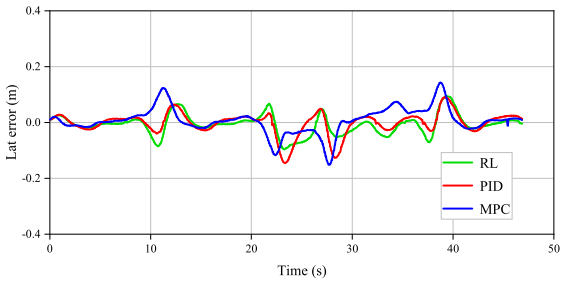


Fig. 17. Lateral control performance in normal curvature path.

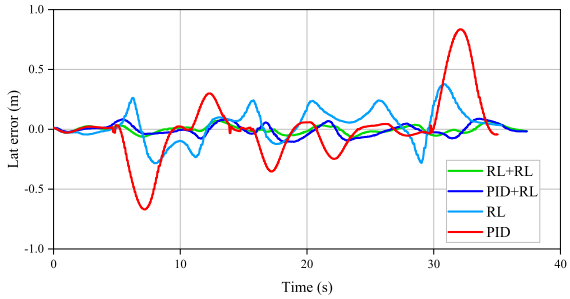


Fig. 18. Lateral control performance along a high-curvature path.

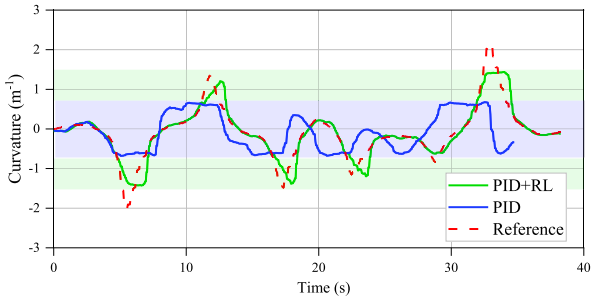


Fig. 19. Lateral control performance along a high-curvature reference path.

vehicle, we conducted testing and validation of the path tracking algorithm in an outdoor parking lot.

As shown in Fig. 16, two path planning experiments were conducted: one for normal curvature scenarios (<1.0 m^{-1}) that do not require deceleration and the other for scenarios with high-curvature that necessitate deceleration (<2.0 m^{-1}).

The RL controller, PID controller, and MPC controller all achieved the expected results in the simulation experiments. Therefore, they were deployed on the vehicle's onboard computing unit for real-world validation. To address the control issues arising from the different response characteristics between the simulation model and real vehicle, the yaw rate output by the network was mapped to real control values using a fuzzy table for the lateral control. Meanwhile, for the longitudinal control, without altering the desired speed from the speed planner, the parameters of the longitudinal controller were adjusted to achieve effective speed tracking.

First, the lateral control algorithm was validated under normal curvature path conditions, where the vehicle maintained a constant maximum desired speed of 1m/s. Fig. 17 shows the corresponding lateral control results. It can be observed that in the real-world scenario, the PID controller achieved lateral deviations of less than 0.15 m, the maximum lateral deviation by the MPC controller is around 0.15 m, while the RL algorithm was less than 0.1 m. It is noteworthy that in real test, the RL time consumption is consistently less than 0.05 s, whereas the MPC exceeds 0.1 s at certain moments.

Then, comprehensive testing of lateral control and longitudinal speed planning was conducted in high-curvature scenarios. Fig. 18 presents the performance of different algorithms. It can be observed that without speed planning, the PID controller results in a maximum lateral error larger than 0.6 m. When combined with longitudinal speed planning, the lateral error was reduced to 0.2 m. When replacing the lateral PID control with RL control, the error further decreased to 0.1 m. All these verify the effectiveness of the proposed adaptive path-tracking controller embedded with RL and preview model.

When navigating sharp curves, the lateral controller's steering capability has an upper limit. This means that there is a threshold for the equivalent curvature that can be achieved at a given speed. If this threshold is lower than the curvature of the reference path, the lateral deviation of the autonomous vehicle will significantly increase. The speed planner addresses this by replanning the maximum desired speed to increase the equivalent curvature, thereby reducing the lateral deviation.

Fig. 19 compares the equivalent curvature with the reference path curvature before and after using the longitudinal speed planner with the PID controller. It can be observed that the speed planner reduces the desired speed. The equivalent curvature was increased from 0.9 m^{-1} to 1.6 m^{-1}, representing an overall increase of 78% (as illustrated by the green area in Fig. 19). It allows the autonomous vehicle to better follow the reference path.

VI. Conclusion

This study addressed the issue of path tracking control for an autonomous vehicle in complex environments, particularly curvy roads. We introduced an adaptive controller using RL to achieve precise path tracking performances through integrating a lateral controller, a lateral adaptive corrector, and a longitudinal speed planner. Experimental validations of the proposed adaptive controller were conducted via simulations and real-world tests. The training results, incorporating the lateral Pre-TD3 algorithm and the RI-SAC method for speed planning,

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on October 28,2026 at 12:11:49 UTC from IEEE Xplore. Restrictions apply.

demonstrated faster convergence than those existing RL methods. The control performance in all test scenarios maintained an accuracy within 0.1 m, outperforming the prevalent controllers. The involved computational cost was less than half of that required by model-based controllers such as MPC.

For future work, it needs to extend the developed controller to different types of vehicles such as medium-sized passenger cars or even a multi-body tractor-trailer vehicle, and address path tracking challenges in more complex urban and highway settings.

REFERENCES

[1] M. Begum, G. Raja, and M. Guizani, "AI-based sensor attack detection and classification for autonomous vehicles in 6G-V2X environment," *IEEE Trans. Veh. Technol.*, vol. 73, no. 4, pp. 5054–5063, Apr. 2024.

[2] H. Liang, L. Zhu, and F. R. Yu, "Collaborative edge intelligence service provision in blockchain empowered urban rail transit systems," *IEEE Internet Things J.*, vol. 11, no. 2, pp. 2211–2223, Jan. 2024.

[3] Z. Xu, C. Wang, T. Jiang, Z. Liu, and N. Zheng, "Impediments to environmental awareness in autonomous driving systems and its effect on user adoption," *IEEE Trans. Intell. Veh.*, early access, Mar. 08, 2024, doi: 10.1109/TIV.2024.3373773.

[4] H. Taghavifar and S. Rakheja, "A novel terramechanics-based path-tracking control of terrain-based wheeled robot vehicle with matched-mismatched uncertainties," *IEEE Trans. Veh. Technol.*, vol. 69, no. 1, pp. 67–77, Jan. 2020.

[5] M. Corno, G. Panzani, F. Roselli, M. Giorelli, D. Azzolini, and S. M. Savaresi, "An LPV approach to autonomous vehicle path tracking in the presence of steering actuation nonlinearities," *IEEE Trans. Control Syst. Technol.*, vol. 29, no. 4, pp. 1766–1774, Jul. 2021.

[6] L. Rankin, C. D. Crane, and D. G. Armstrong, "Evaluating a PID, pure pursuit, and weighted steering controller for an autonomous land vehicle," *Proc. SPIE*, vol. 3210, pp. 1–13, 1998.

[7] J. M. Snider, "Automatic steering methods for autonomous automobile path tracking," Robotics Institute, Pittsburgh, PA, USA, Tech. Rep. CMU-RITR-09-08, 2009.

[8] Z. Qin, L. Chen, M. Hu, and X. Chen, "A lateral and longitudinal dynamics control framework of autonomous vehicles based on multi-parameter joint estimation," *IEEE Trans. Veh. Technol.*, vol. 71, no. 6, pp. 5837–5852, Jun. 2022.

[9] Y. Tian, Q. Yao, P. Hang, and S. Wang, "A gain-scheduled robust controller for autonomous vehicles path tracking based on LPV system with MPC and h∞," *IEEE Trans. Veh. Technol.*, vol. 71, no. 9, pp. 9350–9362, Sep. 2022.

[10] Y. Li, L. Zhu, H. Wang, F. R. Yu, T. Tang, and D. Zhang, "Joint security and resources allocation scheme design in edge intelligence enabled CBTCs: A two-level game theoretic approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 13948–13961, Dec. 2023.

[11] L. Zhu, C. Shen, X. Wang, H. Liang, H. Wang, and T. Tang, "A learning based intelligent train regulation method with dynamic prediction for the metro passenger flow," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 3935–3948, Apr. 2023.

[12] Y. Shan, B. Zheng, L. Chen, L. Chen, and D. Chen, "A reinforcement learning-based adaptive path tracking approach for autonomous driving," *IEEE Trans. Veh. Technol.*, vol. 69, no. 10, pp. 10581–10595, Oct. 2020.

[13] L. Chen, X. Hu, B. Tang, and Y. Cheng, "Conditional DQN-based motion planning with fuzzy logic for autonomous driving," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 4, pp. 2966–2977, Apr. 2022.

[14] S. Dixit, U. Montanaro, M. Dianati, A. Mouzakitis, and S. Fallah, "Integral MRAC with bounded switching gain for vehicle lateral tracking," *IEEE Trans. Control Syst. Technol.*, vol. 29, no. 5, pp. 1936–1951, Sep. 2021.

[15] A. Artuñedo, J. Villagra, and J. Godoy, "Jerk-limited time-optimal speed planning for arbitrary paths," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 8194–8208, Jul. 2022.

[16] Q. Wang, B. Ayalew, and T. Weiskircher, "Predictive maneuver planning for an autonomous vehicle in public highway traffic," *IEEE Trans. Intell. Transp. Syst.*, vol. 20, no. 4, pp. 1303–1315, Apr. 2019.

[17] G. G. M. Hoffmann, C. J. Tomlin, M. Montemerlo, and S. Thrun, "Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and racing," in *Proc. IEEE 2007 Amer. Control Conf.*, 2007, pp. 2296–2301.

[18] C. Hu, Y. Chen, and J. Wang, "Fuzzy observer-based transitional path-tracking control for autonomous vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 5, pp. 3078–3088, May 2021.

[19] X. Lin, Y. Tang, and B. Zhou, "Improved model predictive control path tracking strategy based an online updating algorithm with cosine similarity and a horizon factor," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 12429–12438, Aug. 2022.

[20] J. Fu, F. Tian, T. Chai, Y. Jing, Z. Li, and C.-Y. Su, "Motion tracking control design for a class of nonholonomic mobile robot systems," *IEEE Trans. Syst. Man Cybern.: Syst.*, vol. 50, no. 6, pp. 2150–2156, Jun. 2020.

[21] G. Park and S. B. Choi, "A model predictive control for path tracking of electronic-four-wheel drive vehicles," *IEEE Trans. Veh. Technol.*, vol. 70, no. 11, pp. 11352–11364, Nov. 2021.

[22] Q. Fan, W. Chen, W. Huang, L. Xie, K. Bi, and Y. Zhu, "Combined magnetic field decoupling and disturbance rejection control of microrobots based on extended state observer," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 4032–4039, Apr. 2022.

[23] S. Xu and H. Peng, "Design, analysis, and experiments of preview path tracking control for autonomous vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 1, pp. 48–58, Jan. 2020.

[24] K. Kritayakirana and J. Christian Gerdes, "Using the centre of percussion to design a steering controller for an autonomous race car," *Veh. Syst. Dyn.*, vol. 50, pp. 33–51, 2012.

[25] P. Falcone, F. Borrelli, J. Asgari, H. E. Tseng, and D. Hrovat, "Predictive active steering control for autonomous vehicle systems," *IEEE Trans. Control Syst. Technol.*, vol. 15, no. 3, pp. 566–580, May 2007.

[26] P. Falcone, "MPC-based yaw and lateral stabilisation via active front steering and braking," *Veh. Syst. Dyn.*, vol. 46, pp. 611–628, 2008.

[27] Y. Gao, "A tube-based robust nonlinear predictive control approach to semiautonomous ground vehicles," *Veh. Syst. Dyn.*, vol. 52, pp. 802–823, 2014.

[28] G. Tagne, R. Talj, and A. Charara, "Higher-order sliding mode control for lateral dynamics of autonomous vehicles, with experimental validation," in *Proc. IEEE Intell. Veh. Symp.*, 2013, pp. 678–683.

[29] Z. Chu, C. Wu, and N. Sepehri, "Automated steering controller design for vehicle lane keeping combining linear active disturbance rejection control and quantitative feedback theory," *Proc. Inst. Mech. Eng., Part I: J. Syst. Control Eng.*, vol. 232, pp. 937–948, 2018.

[30] M. Rokonuzzaman, N. Mohajer, and S. Nahavandi, "Human-tailored data-driven control system of autonomous vehicles," *IEEE Trans. Veh. Technol.*, vol. 71, no. 3, pp. 2485–2500, Mar. 2022.

[31] Q. Zhang et al., "TrajGen: Generating realistic and diverse trajectories with reactive and feasible agent behaviors for autonomous driving," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 24474–24487, Dec. 2022.

[32] J. Chen, C. Wu, G. Yu, D. Narang, and Y. Wang, "Path following of wheeled mobile robots using online-optimization-based guidance vector field," *IEEE/ASME Trans. Mechatron.*, vol. 26, no. 4, pp. 1737–1744, Aug. 2021.

[33] Y. Tian, X. Cao, K. Huang, C. Fei, Z. Zheng, and X. Ji, "Learning to drive like human beings: A method based on deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 6357–6367, Jul. 2022.

[34] C. Ginerica, M. Zaha, F. Gogianu, L. Busoniu, B. Trasnea, and S. Grigorescu, "Observenet control: A vision-dynamics learning approach to predictive control in autonomous vehicles," *IEEE Robot. Automat. Lett.*, vol. 6, no. 4, pp. 6915–6922, Oct. 2021.

[35] P. Rousseas, C. P. Bechlioulis, and K. J. Kyriakopoulos, "Optimal motion planning in unknown workspaces using integral reinforcement learning," *IEEE Robot. Automat. Lett.*, vol. 7, no. 3, pp. 6926–6933, Jul. 2022.

[36] X. Yu, "Self-adaptive LSAC-PID approach based on lyapunov reward shaping for mobile robots," *J. Shanghai Jiaotong Univ. (Sci.)*, pp. 1–18, 2023.

[37] P. Lin, Y. S. Quan, J. H. Yang, C. C. Chung, and M. Tsukada, "Safety tunnel-based model predictive path-planning controller with potential functions for emergency navigation," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 3974–3985, Apr. 2023.

[38] Z. Li, J. Zhou, X. Li, X. Du, L. Wang, and Y. Wang, "Continuous control for moving object tracking of unmanned skid-steered vehicle based on reinforcement learning," in *Proc. Int. Conf. Unmanned Syst.*, 2020, pp. 456–461.

[39] Y. S. Shao, C. Chen, S. Kousik, and R. Vasudevan, "Reachability-based trajectory safeguard (RTS): A safe and fast reinforcement learning safety layer for continuous control," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 3663–3670, Apr. 2021.

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on October 28,2026 at 12:11:49 UTC from IEEE Xplore. Restrictions apply.

[40] M. Liu, F. Zhao, J. Yin, J. Niu, and Y. Liu, "Reinforcement-tracking: An effective trajectory tracking and navigation method for autonomous urban driving," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 6991–7007, Jul. 2022.
[41] H. Liang, L. Zhu, F. R. Yu, and X. Wang, "A cross-layer defense method for blockchain empowered CBTC systems against data tampering attacks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 1, pp. 501–515, Jan. 2023.
[42] J. Schulman, "Proximal policy optimization algorithms," Jul. 2017, *arXiv:1707.06347*.
[43] T. P. Lillicrap, "Continuous control with deep reinforcement learning," Sep. 2015, *arXiv:1509.02971*.
[44] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1582–1591.
[45] T. Haarnoja, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
[46] N. Heess, G. Wayne, D. Silver, T. Lillicrap, T. Erez, and Y. Tassa, "Learning continuous control policies by stochastic value gradients," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 2944–2952.
[47] G. Huang, X. Yuan, K. Shi, Z. Liu, and X. Wu, "Adaptivity-enhanced path tracking system for autonomous vehicles at high speeds," *IEEE Trans. Intell. Veh.*, vol. 5, no. 4, pp. 626–634, Dec. 2020.



Qi Xia received an M.S. degree from Beihang University, Beijing, China in 2022. He is currently working toward the Ph.D. degree with the School of Transportation Science and Engineering, Beihang University. His research interests include reinforcement learning, path planning, speed planning, and closed-loop tracking control of an intelligent vehicle.



Peng Chen (Member, IEEE) received the Ph.D. degree from Nagoya University, Nagoya, Japan, in 2012. He is currently an Associate Professor with the School of Transportation Science and Engineering, Beihang University, Beijing, China. His research interests include trajectory planning of autonomous vehicles, urban traffic operation and control. He has been an Associate Editor of IEEE TRANSACTIONS ON INTELLIGENT VEHICLES since 2022.



Guoyan Xu received the Ph.D. degree from Beihang University, Beijing, China, in 2003. She is an Associate Professor with the School of Transportation Science and Engineering, Beihang University. Her research interests include intelligent vehicle decision and control, intelligent vehicle environment perception and reinforcement learning.



Haodong Sun received the Ph.D. degree from the Beijing University of Technology, Beijing, China, in 2023. He is currently with the Beijing Municipal Institute of City Planning and Design, Beijing, China. His research interests include urban transportation planning, travel behavior and intelligent transportation systems.



Liang Li received the M.S. degree from the Beijing University of Technology, Beijing, China, in 2016. He is currently with the Beijing Municipal Commission of Planning and Natural Resources, and the Deputy Director of the Comprehensive Transportation Department. His research interests include comprehensive transportation planning and management, intelligent transportation systems for planning applications, and new infrastructure related to smart transportation.



Guizhen Yu received the Ph.D. degree from Jilin University, Changchun, China, in 2003. He is currently a Professor with the School of Transportation Science and Engineering, Beihang University, Beijing, China. His research interests include intelligent vehicle, urban traffic operation, and intelligent transportation systems.

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on October 28,2026 at 12:11:49 UTC from IEEE Xplore. Restrictions apply.